

1 What you want conceptually

MobileNetV2 structure:

```
input
  ↓
features (Conv + BN + ReLU + InvertedResiduals) ← heavy, expensive
  ↓
avgpool
  ↓
classifier (Linear) ← small
```

Your idea:

- ♦ Quantize **features** to INT8
- ♦ Keep **classifier** in FP32 (trainable / flexible)

This is a **very common hybrid strategy**.

2 What PyTorch actually allows

Operation	Supported
Quantize <i>entire</i> MobileNetV2	✓
Quantize only features	⚠ Yes, but carefully
Load quantized features into FP32 module	✗
Fine-tune quantized features	✗
Hybrid INT8 features + FP32 classifier	✓ (inference)

👉 The key rule:

Once a submodule is quantized, it becomes inference-only.

3 Correct Python approach: quantize only features

Step 1 — Load pretrained model

```
import torch
import torch.quantization as tq
from torchvision import models

model = models.mobilenet_v2(
    weights=models.MobileNet_V2_Weights.IMAGENET1K_V1
)
model.eval()
```

Step 2 — Fuse only the feature extractor

TorchVision already knows how to do this:

```
model.features.fuse_model()
```

⚠ Do **not** fuse the classifier.

Step 3 — Assign qconfig only to features

This is the crucial part.

```
model.features.qconfig = tq.get_default_qconfig("qnnpack") # ARM
model.classifier.qconfig = None # keep FP32
```

If you skip this, **everything gets quantized**.

Step 4 — Prepare (observers inserted only in features)

```
tq.prepare(model, inplace=True)
```

Observers are now **only inside features**.

Step 5 — Calibration

```
with torch.no_grad():
    for _ in range(200):
        x = torch.rand(1, 3, 224, 224)
        model(x)
```

Use real images if possible.

Step 6 — Convert

```
tq.convert(model, inplace=True)
```

🎉 Result:

- `model.features` → **INT8**
 - `model.classifier` → **FP32**
-

4 Verifying the result

```
print(model.features[0])
print(model.classifier)
```

You should see things like:

```
QuantizedConvReLU2d(...)
Linear(in_features=1280, out_features=1000)
```

And programmatically:

```
for name, m in model.named_modules():
```

```
if "Quantized" in type(m).__name__:  
    print(name, type(m))
```

5 Exporting for C++ / embedded

✗ You cannot export this as a `state_dict` for C++

Because:

- Quantized features contain packed kernels
 - Your C++ MobileNetV2 FP32 class cannot consume them
-

✓ Correct export: TorchScript

```
example = torch.rand(1, 3, 224, 224)  
scripted = torch.jit.trace(model, example)  
scripted.save("mobilenetv2_features_int8.pt")
```

On embedded (C++):

```
torch::jit::script::Module model =  
    torch::jit::load("mobilenetv2_features_int8.pt");  
  
auto output = model.forward({input}).toTensor();
```

- ✓ No TorchVision
 - ✓ Hybrid INT8 + FP32
 - ✓ Fast inference
 - ✓ Small memory footprint
-

6 Can you still fine-tune the classifier?

● Not inside TorchScript

TorchScript = inference only.

✓ Recommended pattern

1. **Train / fine-tune classifier in FP32**
2. **Freeze it**
3. **Quantize features**
4. **Export TorchScript**

If you want **on-device fine-tuning**, do it **before quantization**.

7 Alternative advanced pattern (if you REALLY need training)

If you need:

- INT8 features
- FP32 classifier
- training on device

👉 **This is not supported in PyTorch today.**

Production systems handle this by:

- Training FP32
- Quantizing only for final deployment
- Never training quantized weights

8 Summary (clear answers)

Question	Answer
Can I quantize only MobileNet features?	✓ Yes
Can classifier stay FP32?	✓ Yes
Can I save this as state_dict?	✗ No
Can I load into C++ FP32 module?	✗ No
Can I deploy via TorchScript?	✓ Yes
Can I fine-tune after quantization?	✗ No